

Модуль безопасности персональных данных

Техническое задание трека · условия и автоматическая проверка решений

Автоматическая проверка включает два вида: (1) проверку качества кода загруженного решения и (2) нагрузочное тестирование развёрнутого решения. Обе проверки обязательны — см. раздел 7.

1. Краткое описание

В Банке в связи с активным развитием генеративного искусственного интеллекта есть много продуктов для клиентов и сотрудников, которые генерируют большое количество запросов к LLM. По требованиям Банка критично соблюдать безопасность персональных данных при обращении к LLM.

Ваша задача: разработать Модуль безопасности персональных данных. Модуль должен уметь определять тип персональных данных, осуществлять их маскирование и последующее демаскирование.

Цель — создать сервис, который может выполнять роль прокси Core-компонента, выдерживать нагрузку обращений к LLM, отвечать быстро и качественно.

Трек подойдёт вам, если вы:

- LLM-инженер / AI Engineer
- Backend- или Frontend-разработчик
- UX/UI-дизайнер
- Продуктовый менеджер или аналитик
- Data Scientist / ML Engineer
- Специалист по банковским процессам или безопасности данных

2. Полное описание

Проблематика

Сегодня все команды, работающие с LLM в Банке, сталкиваются с необходимостью защищать персональные данные, однако каждая команда решает эту задачу самостоятельно. Это создаёт ряд системных проблем:

- Текущие решения разработаны разными командами под свои требования и ограничения.
- Решения недостаточно гибкие с точки зрения настройки маскирования для разных типов персональных данных.
- Демаскирование присутствует лишь частично.
- Сервисы должны быть отказоустойчивыми и держать большую нагрузку.

В результате разрозненные решения разных команд трудно переиспользовать, команды вынуждены тратить лишние ресурсы на создание собственного решения.

Образ решения

Ваша задача — создать приложение, которое качественно и быстро обрабатывает запрос, отправляемый системой-потребителем в LLM, а также ответ, который формирует LLM. В рамках обработки должны быть осуществлены идентификация персональных данных, их маскирование и демаскирование.

Нам важно увидеть:

- насколько хорошо сервис находит информацию, которую нельзя передавать без обработки;
- насколько мало при этом ложных срабатываний;
- сохраняется ли после обработки смысл запроса для AI-модели;
- можно ли управлять правилами для разных систем;
- насколько просто подключить нового потребителя;
- что происходит при ошибках или недоступности отдельных компонентов;
- не попадают ли исходные защищаемые данные в логи и технические метрики.

Архитектура должна позволять добавлять новые правила и сценарии без переделки всего сервиса.

3. Ключевые требования

- Решение должно легко встраиваться в цепочку взаимодействия «система-потребитель — LLM».
- Обязательное покрытие описанных типов персональных данных.
- Строгое соблюдение требований безопасности и приватности: никаких утечек, шифрование, минимизация данных.
- Недопустимость критичных ошибок и соблюдение уровня качества обработки.
- Выполнение SLA на ответы и действия (скорость, доступность, стабильность).

4. Функциональные требования

Сервис должен выявлять и обеспечивать защиту персональных данных в соответствии с действующим законодательством Российской Федерации, применимой нормативной базой и требованиями ЦБ.

4.1. Идентификация типов персональных данных

Необходимо однозначно идентифицировать следующие типы персональных данных:

- | | |
|---|---|
| • ФИО | • Дата рождения |
| • Место рождения | • Серия и номер паспорта |
| • Гражданство | • Орган, выдавший паспорт |
| • Код подразделения | • Дата выдачи паспорта |
| • Серия и номер водительского удостоверения | • Адрес (в т.ч. отдельно: страна, индекс, город, улица, дом, квартира и т.п.) |
| • Email | • Номер телефона |

- ИНН
- CVV-код
- Имя держателя карты
- Номер платёжной банковской карты
- Пин-код карты
- Необходимо проработать механизм определения принадлежности данных именно к персональным: например, упоминание поэта Александра Пушкина не является персональными данными, как и адрес отделения Банка.
- Возможность расширения списка идентифицируемых персональных данных.
- Идентификация не должна зависеть от регистра.
- Логирование выявленных типов персональных данных в тесте по каждому запросу.

4.2. Маскирование

Необходимо учесть разные вариации написания персональных данных, например:

- регистр;
- разное расположение года, месяца и даты (например, мм.дд.гггг, гггг.дд.мм);
- написание даты текстом, а не числами;
- разделяющие слова — «серия xxxx номер xxxxxx» и т.п.

4.3. Демаскирование

Преобразование замаскированного текста в незамаскированный с сохранением местоположения знаков.

4.4. Общие требования

- Подробное логирование на каждом этапе работы сервиса.
- Метрики для контроля Latency, RPS, TPS (tokens per second).
- Наличие ограниченного списка систем, которые имеют возможность обращаться в сервис.
- Гибкая настройка:
 - списка систем (вкл/откл возможности обращения в модуль);
 - перечня типов персональных данных для идентификации и маскирования в зависимости от системы;
 - наличия демаскирования в зависимости от системы;
 - краткая инструкция по настройке (не более 5 предложений).
- Высокое качество идентификации/маскирования/демаскирования — 95%.
- Обработка крупных массивов текста — до 100 000 токенов.

5. Нефункциональные требования

- **Безопасность:** все данные обрабатываются в защищённом контуре, соответствие внутренним стандартам ИБ банка.
- **Надёжность:** корректная обработка ошибок API, деградация функционала при недоступности сервисов, понятные сообщения пользователю.
- **Производительность:** Latency — не более 1 секунды, RPS — 1000.
- **Масштабируемость:** архитектура допускает добавление настроек для новых типов ПД и увеличение RPS при сохранении Latency.

6. Дополнительные плюсы

- Дополнительная реализация токенизации/детокенизации либо замены синтетическими данными.
- Гибкая настройка вида маскирования/токенизации в зависимости от системы.
- Возможность держать RPS 2000 при Latency не более 1 секунды.
- Идентификация документов, удостоверяющих личность, кроме паспорта РФ.
- Маскирование только при наличии нескольких однозначно идентифицированных типов ПД (например, «пин-код карты» — не маскируем; «пин-код карты + номер карты» — маскируем).
Возможность изменения этих настроек.

7. Автоматическая проверка решений

Обе проверки обязательны. Решение, не прошедшее автоматическую проверку качества кода, а также не развёрнутое для нагрузочного тестирования, **не будет учтено**.

7.1. Автоматическая проверка качества кода

Как только вы загружаете решение **в формате zip-файла (и только zip-файла!)**, автоматически запускается проверка качества кода. **Без её прохождения решение не будет учтено.**

Как правильно упаковать решение в zip

- Упаковывайте **только исходный код** решения.
- **Не добавляйте объёмные файлы** — датасеты, дампы, архивы, медиа, скомпилированные бинарники.
- **Не упаковывайте build-директории и зависимости**, например:
 - `node_modules`, `.venv`, `venv`, `env` — каталоги зависимостей;
 - `target`, `build`, `dist`, `out`, `bin`, `obj` — каталоги сборки;
 - `.git`, `.idea`, `__pycache__`, `coverage` — служебные каталоги.
- Один архив в формате **.zip** разумного размера, с исходным кодом в корне или в понятной структуре.

Совет: чем чище и легче архив, тем быстрее и надёжнее проходит проверка. Лишние каталоги зависимостей и сборки не нужны — проверяется именно ваш исходный код.

7.2. Нагрузочное тестирование решений

В **обязательном порядке** потребуется **развернуть ваше решение**, реализующее контракт API (эндпоинт `POST /process`). Проверяющая система подаёт нагрузку по эталонному датасету и оценивает точность маскирования и демаскирования, а также поведение под нагрузкой.

Ниже приведены **контракт API** (Приложение А) и **инструкция по прохождению проверки** (Приложение В). Реализуйте контракт и держите решение доступным по указанному URL на момент прогона.

Приложение А. Контракт API «/process»

Единый контракт, который команды-участники **должны** реализовать на своём сервисе. Проверяющая система обращается к этому эндпоинту для нагрузочного тестирования.

Схема запроса/ответа

```
POST /process
Content-Type: application/json
{ "payload": "&lt;строка&gt;", "payload_id": "&lt;идентификатор&gt;" }
  → 200
{ "result": "&lt;строка&gt;" }
```

Логика (одинакова для всех):

- Эндпоинт **один** — обрабатывает и маскирование, и демаскирование.
- Направление определяется по `payload_id` — используйте его как ключ корреляции «маскирование → демаскирование».
- Первый запрос с новым `payload_id` — маскирование (`payload` = исходная строка) → верните маску и запомните соответствие.
- Второй запрос с тем же `payload_id` — демаскирование (`payload` = ваша маска) → верните исходную строку.
- Ответ всегда: `{"result": "..."}.`

Полная OpenAPI-спецификация (файл `process_api.yaml`)

openapi: 3.0.3

info:

title: Модуль безопасности ПД – контракт /process

version: "1.0.0"

description: |

Единый контракт, который команды-участники **ДОЛЖНЫ** реализовать на своём сервисе.

Система проверки AlfaSonar обращается к этому эндпоинту для нагрузочного тестирования и оценки точности маскирования/демаскирования персональных данных.

ВАЖНО (логика проверки одинакова для всех команд):

- * Эндпоинт один – `POST /process`. Он обрабатывает и маскирование, и демаскирование.
- * Проверка идёт парами по каждому элементу датасета и коррелируется полем `payload_id`:
 - 1) ПРЯМАЯ проверка (маскирование): приходит первый запрос с новым `payload_id`, `payload` = исходная строка. Сервис **ДОЛЖЕН** вернуть замаскированную строку и запомнить соответствие по `payload_id`.
 - 2) ОБРАТНАЯ проверка (демаскирование): приходит второй запрос с **ТЕМ ЖЕ** `payload_id`, `payload` = ранее возвращённая вами замаскированная строка. Сервис **ДОЛЖЕН** вернуть исходную (демаскированную) строку.
- * `payload_id` уникален для каждого элемента в рамках прогона – используйте его как ключ корреляции «маскирование → демаскирование».
- * Ответ всегда: `{"result": "<строка>"}`.

servers:

- url: `https://your-solution.example.com`
description: HTTPS (self-signed допускается – проверка идёт с `ssl_verify=false`)
- url: `http://your-solution.example.com`
description: HTTP также поддерживается

paths:

```
/process:
  post:
    operationId: process
```

```

summary: Маскирование/демаскирование строки (коррелируется по payload_id)
requestBody:
  required: true
  content:
    application/json:
      schema:
        $ref: "#/components/schemas/ProcessRequest"
responses:
  "200":
    description: Успешная обработка
    content:
      application/json:
        schema:
          $ref: "#/components/schemas/ProcessResponse"
  "429":
    description: |
      Слишком много запросов. Допустимо под нагрузкой. Проверяющая система применит
      ожидание и ретрай; если 429 не проходит – это значимый результат (решение не держит нагрузку).
  "4XX":
    description: Ошибка запроса
  "5XX":
    description: Внутренняя ошибка сервиса

components:
  schemas:
    ProcessRequest:
      type: object
      required: [payload, payload_id]
      properties:
        payload:
          type: string
          description: |
            Строка для обработки. На прямом шаге – исходный текст с ПД; на обратном шаге –
            ваша же ранее возвращённая замаскированная строка (тот же payload_id).
          example: "Клиент Иванов Иван Иванович, паспорт 4509 123456"
        payload_id:
          type: string
          description: Идентификатор корреляции. Один и тот же для пары маскирование→демаскирование.
          example: "8a77d363c7c044b49b41d7b8a448243a"
    ProcessResponse:
      type: object
      required: [result]
      properties:
        result:
          type: string
          description: Результат обработки (замаскированная строка на прямом шаге, исходная – на обратном).
          example: "Клиент И. И. И., паспорт 45** ****56"

```

Приложение В. Инструкция по нагрузочному тестированию

Ваш сервис проверяется системой нагрузочного тестирования: она подаёт нагрузку по эталонному датасету и оценивает точность маскирования и демаскирования. **Условия одинаковы для всех команд.**

Что нужно реализовать

- Один HTTP-эндпоинт `POST /process` по контракту из Приложения А.
- Сервис можно развернуть на **http или https** (в т.ч. с самоподписанным сертификатом) — проверка идёт с `ssl_verify=false`, ошибок сертификата не будет.

Логика проверки (пошагово)

Проверка идёт по каждому элементу датасета парой запросов с одним `payload_id`:

- **Прямая проверка (маскирование).** Система шлёт исходную строку с новым `payload_id` → ваш сервис возвращает маску и запоминает соответствие. Результат сравнивается с эталонной маской (метрика — нормированное span-based расстояние Левенштейна, [0..1]).
- **Обратная проверка (демаскирование).** Система шлёт вашу же маску с тем же `payload_id` → ваш сервис возвращает исходную строку. Результат сравнивается с исходной строкой из датасета.

Проверяются категории ПД из ТЗ (ФИО, даты, паспорт, гражданство, орган выдачи, код подразделения, в/у, адрес, email, телефон, ИНН, номер карты, CVV, ПИН, имя держателя и сложные предложения).

Требования к надёжности (обязательно учесть)

- **Таймаут одного запроса — 10 секунд.** Ответ дольше считается неответом.
- **Ретраи — до 2** на каждый запрос (итого до 3 попыток). Делайте эндпоинт идиempотентным по `payload_id`.
- **429 (Too Many Requests)** допустим под нагрузкой: система применит ожидание и ретрай (учитывается `Retry-After`). Если 429 сохраняется — это значимый результат (решение не держит нагрузку) и попадёт в метрики.
 - Если 429 не проходит на маскировании — демаскирование по этому элементу не запрашивается; если на демаскировании — учитывается, и проверка продолжается.
- **Стоп при недоступности:** после **5 невалидных запросов подряд** прогон останавливается (429 невалидным не считается и счётчик не сбрасывает; успешный ответ сбрасывает счётчик).
- **Нагрузка:** по умолчанию прогон идёт по всем категориям ~5 минут с целевым **RPS 1000**; элементы датасета переиспользуются. Целевое значение `latency` — не более 1 с (ориентир, не жёсткий лимит).

Чек-лист «как пройти успешно»

- ☐ Реализован `POST /process` строго по контракту (`{payload, payload_id} → {result}`).
- ☐ Маскирование сохраняет соответствие по `payload_id`; повторный запрос с тем же `id` возвращает исходную строку.
- ☐ Ответ отдаётся быстро и стабильно под нагрузкой (RPS 1000+).
- ☐ Эндпоинт идиempотентен (безопасен к ретраям), корректно возвращает 429 при перегрузке.
- ☐ Сервис доступен по указанному URL (`http` или `https`) на момент прогона.

Самопроверка формата контракта

Эталонная реализация **формата** контракта доступна для самопроверки — она демонстрирует только формат обмена (правила маскирования берутся из ТЗ и одинаковы для всех):

```
curl -X POST https://process-test.holydev.space/process \
  -H "Content-Type: application/json" \
  -d '{"payload":"тестовая строка","payload_id":"selfcheck-1"}'
```